Estrutura de Dados

Aula 03 – Recursividade e Análise de Algoritmos

ibmec

Quem sou eu?

Professor: Luís Aramis dos Reis Pinheiro.

- **Doutorado e Mestrado** em **Ciências Mecânicas** – **UnB** – **CAPES 7**
- **Graduação** em Licenciatura em Física – **UNIFAP**

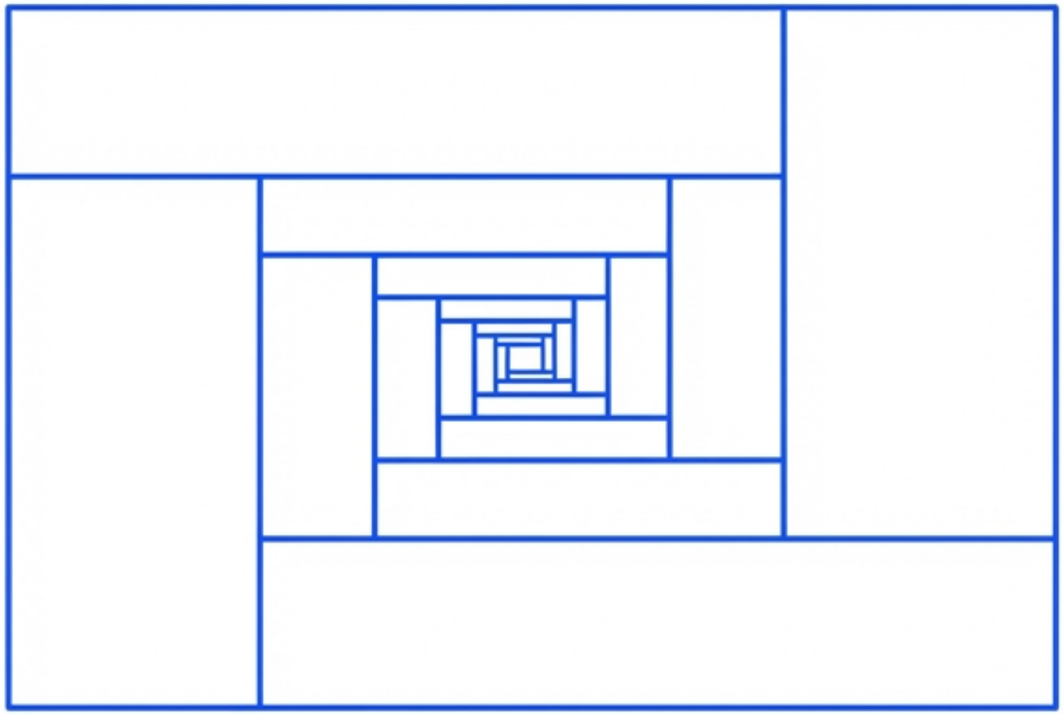
aramisrp@gmail.com

(96) 99907-5819

@l_aramis

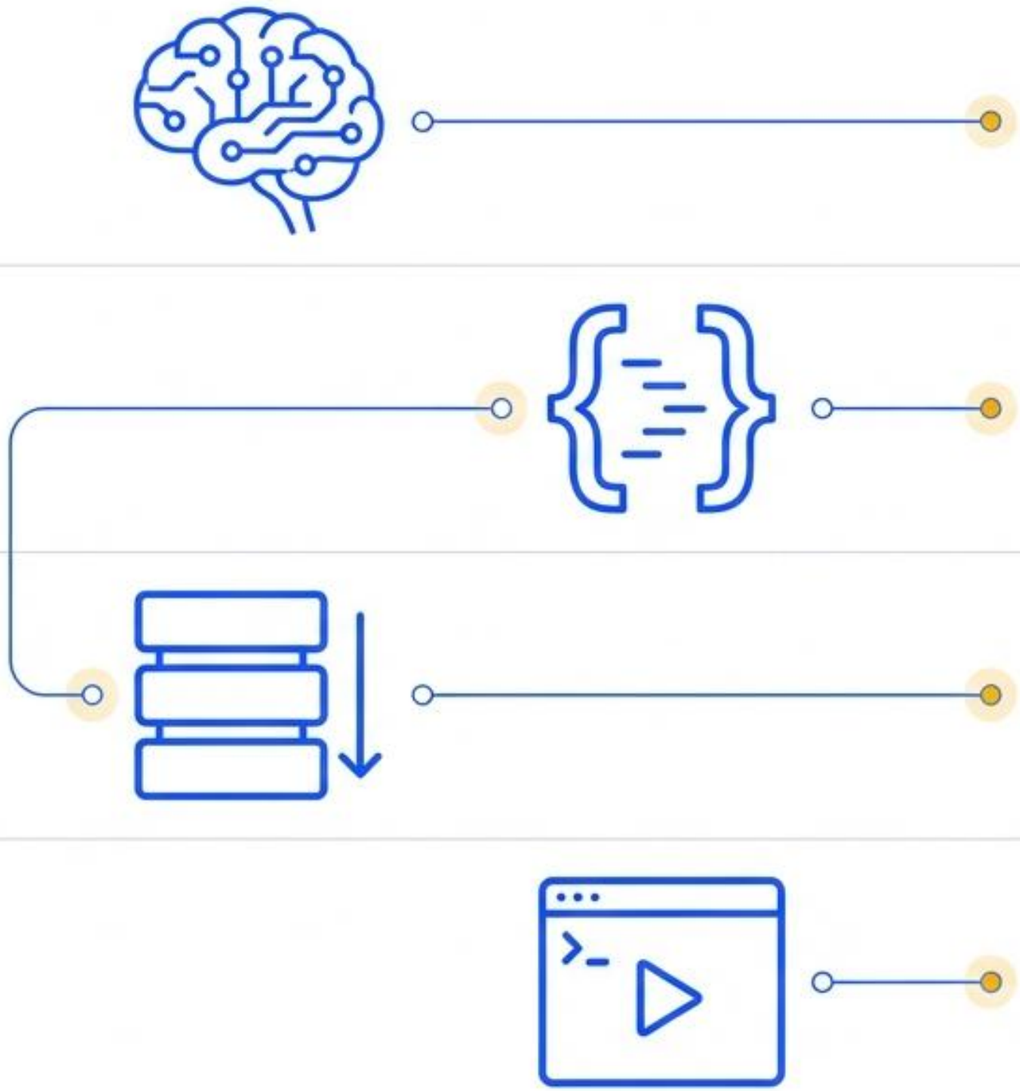
Recursividade e Análise de Algoritmos

Abordagem Iterativa vs. Recursiva



Disciplina: Estruturas de Dados (IBM3114)
Aula 03 | Tempo 01

Objetivos de Aprendizagem (00:00 - 00:50)



Diferenciar abordagens Iterativas de Recursivas.

Implementar funções recursivas (Anatomia do Código).

Visualizar a Pilha de Execução (Call Stack).

Live Coding: Fatorial Iterativo vs. Recursivo.

Definição do Problema: Fatorial (n!)

$$n! = n \times (n-1) \times \ldots \times 1$$

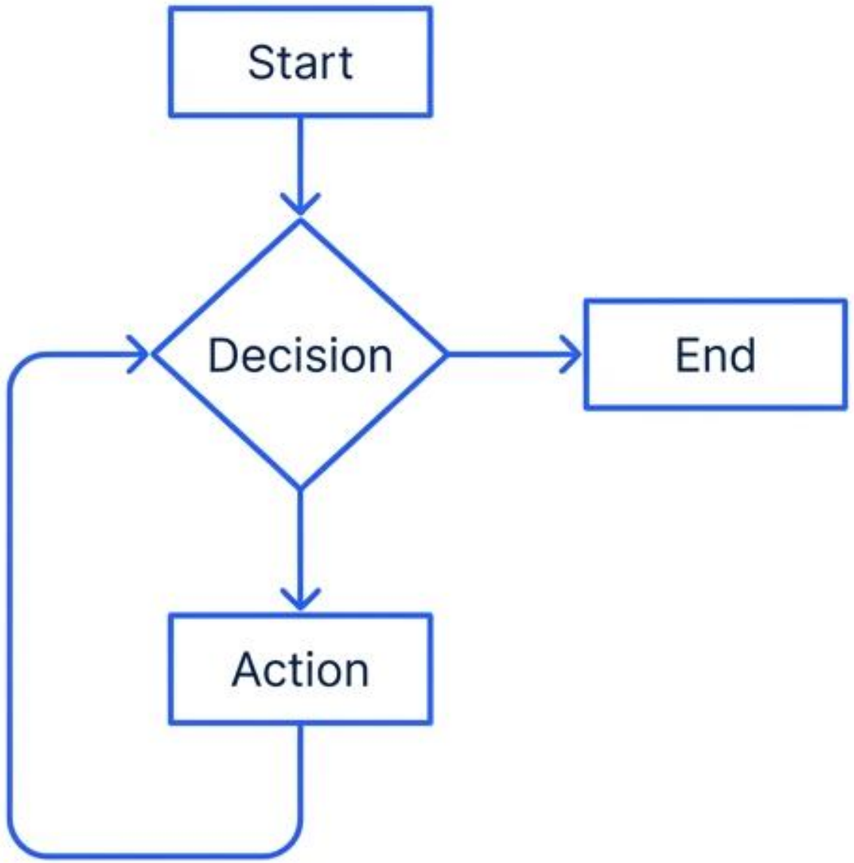
Caso especial: 0! = 1

Exemplo: 5! = 120

A Solução Iterativa (Loops)

Características

- **Lógica**: Repetição explícita controlada pelo programador.
- **Controle**: Comandos `for` ou `while`.
- **Memória**: Complexidade de Espaço O(1) (Constante).



Check de Aprendizado

Desafio: Escreva `fatorial_iterativo(n)` no seu editor.
Tempo: 2 minutos.

Pergunta: A memória RAM gasta aumenta se n for 10 ou 10.000?

Round 1: A Abordagem Iterativa

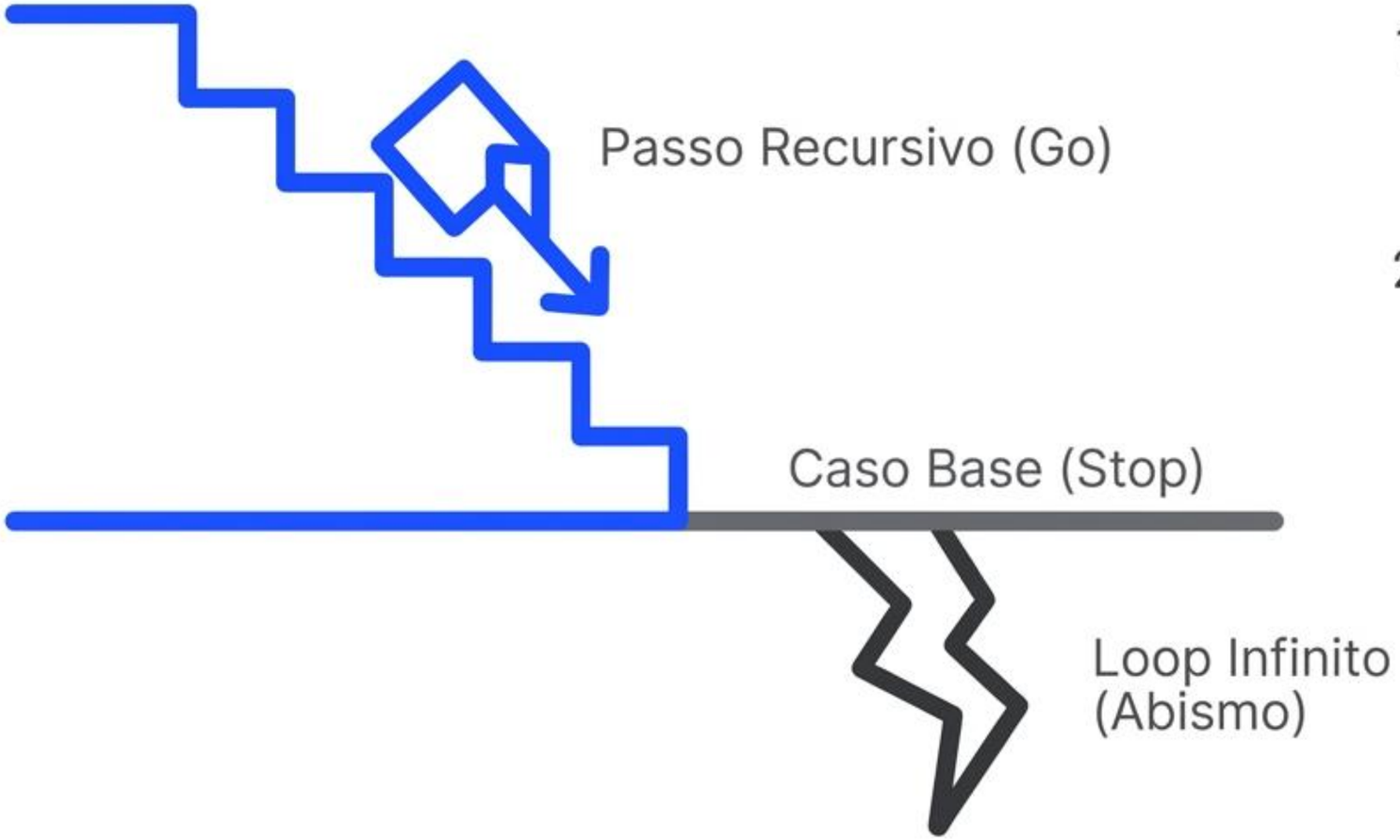
```python
def fatorial_iterativo(n):
    resultado = 1
    for i in range(1, n + 1):
        resultado *= i
    return resultado
```

- **Estratégia:** Uso de um loop explícito (`for` ou `while`).
- **Memória:** $O(1)$ (Constante).
- **Características:** Direto, eficiente e baixo consumo de memória.

A Solução Recursiva

- Definição: Uma função que chama a si mesma.
- Lógica: “Divisão e Conquista” (quebrar em subproblemas menores).
- Delegação: O controle do loop passa para o interpretador.

Anatomia da Recursão (Regra de Ouro)



1. Caso Base:
   `if n < 2: return 1`

2. Passo Recursivo:
   `return n * fat(n-1)`

recursão

Significado de Recursão

substantivo feminino

Ação de recorrer; recurso.

Etimologia (origem da palavra ***recursão***). Do latim recursione.

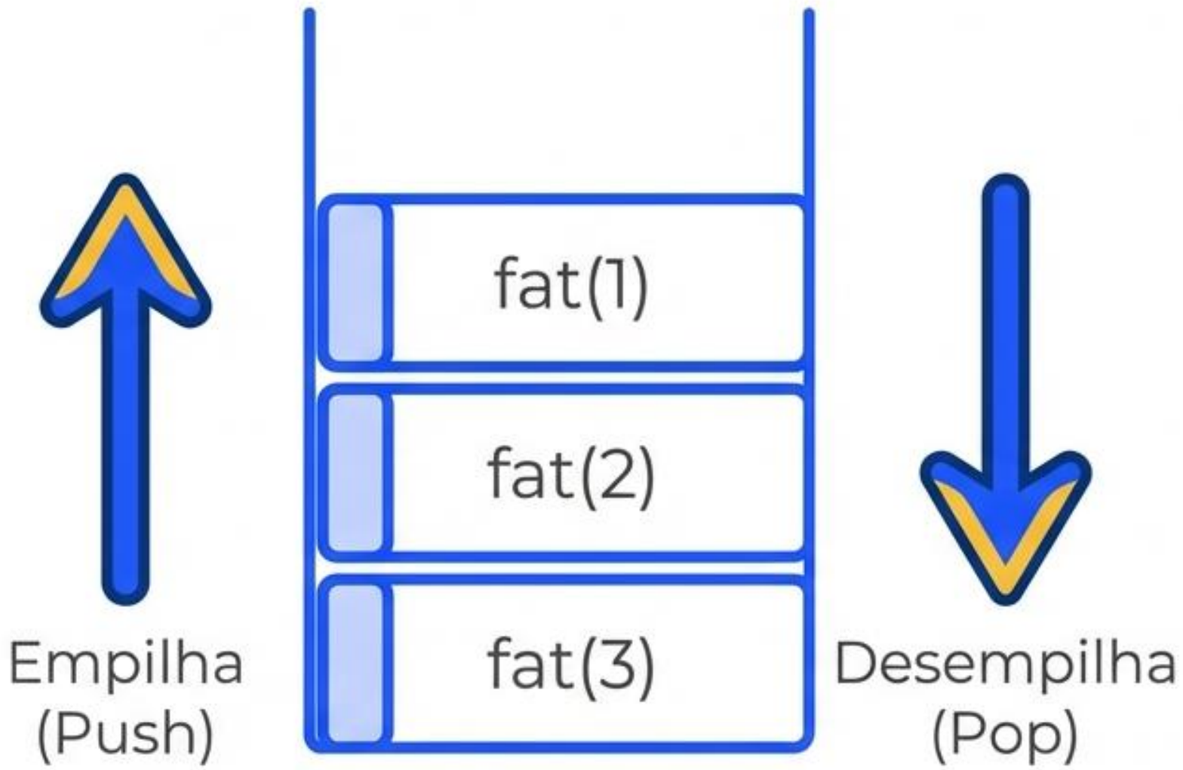
Implementação em Python

```python
def fatorial_recursivo(n):
    if n < 2:
        return 1
    else:
        return n * fatorial_recursivo(n - 1)
```

Caso Base

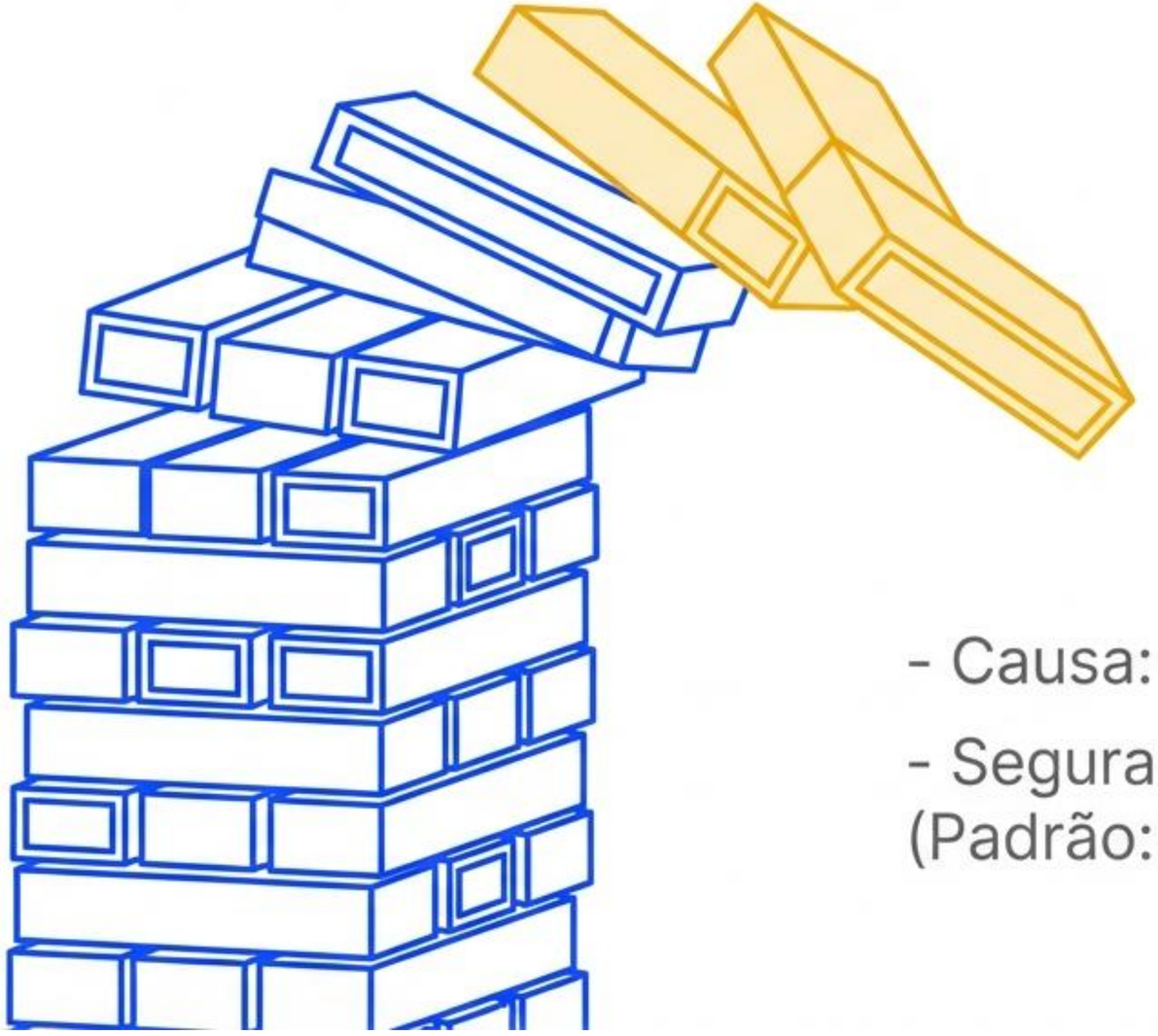
Passo Recursivo

O Custo Invisível: Call Stack



1. Cada chamada cria um novo Frame na memória.
2. O computador pausa a função atual.
3. Empilha o novo contexto no topo.

O Limite: Stack Overflow



```
RecursionError: maximum
recursion depth exceeded
```

- Causa: Estouro da capacidade da Pilha.
- Segurança: Python limita a profundidade (Padrão: 1000 frames).

Laboratório: Prática Imediata

Atividade
- Implementar as versões Iterativa e Recursiva.

Desafio
- Comparar visualmente o código.

Próximo Passo (Bloco 02)
- Medição de Tempo e Complexidade Big O.

Análise de Algoritmos e Complexidade

Contexto: Aula 03 - Tempo 2

Disciplina: Estruturas de Dados

Foco: Medindo a eficiência computacional

A Ilusão do Tempo de Relógio

$\neq$

- **Variabilidade de Hardware:** Supercomputador vs. Laptop
- **Interferência:** Processos em segundo plano (OS)
- **Inconsistência:** Resultados variáveis para entradas pequenas

Necessidade: Uma métrica independente da máquina

Análise Assintótica e Crescimento

$$T(n) = 3n^2 + 20n$$

Ignorar constantes e termos menores

- Função de custo baseada no tamanho da entrada (n).
- Foco na taxa de crescimento no limite ($n \to \infty$).
- Abstração de hardware e detalhes de implementação.

Análise Prática: Linear vs. Quadrático

Linear

```
for i in range(n):
    print(i)
```

Crescimento Linear (n)

Quadrático

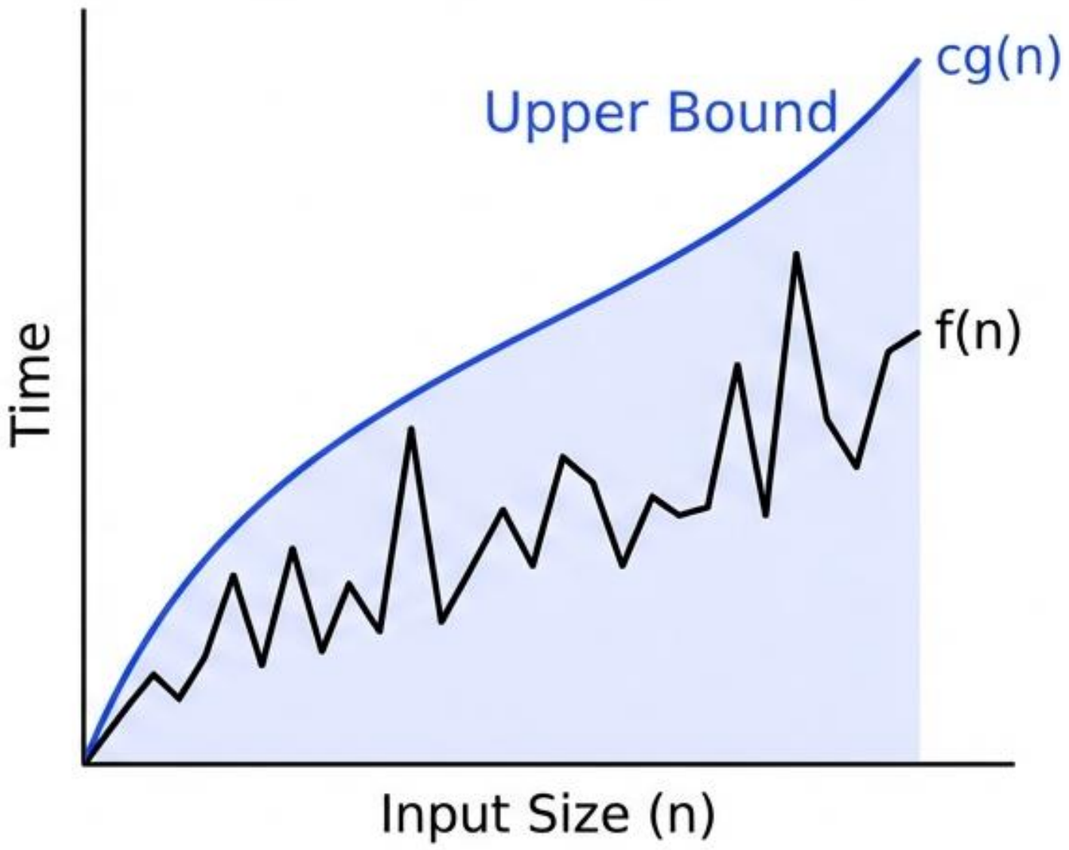
```
for i in range(n):
    for j in range(n):
        print(i, j)
```

Crescimento Quadrático (n^2)

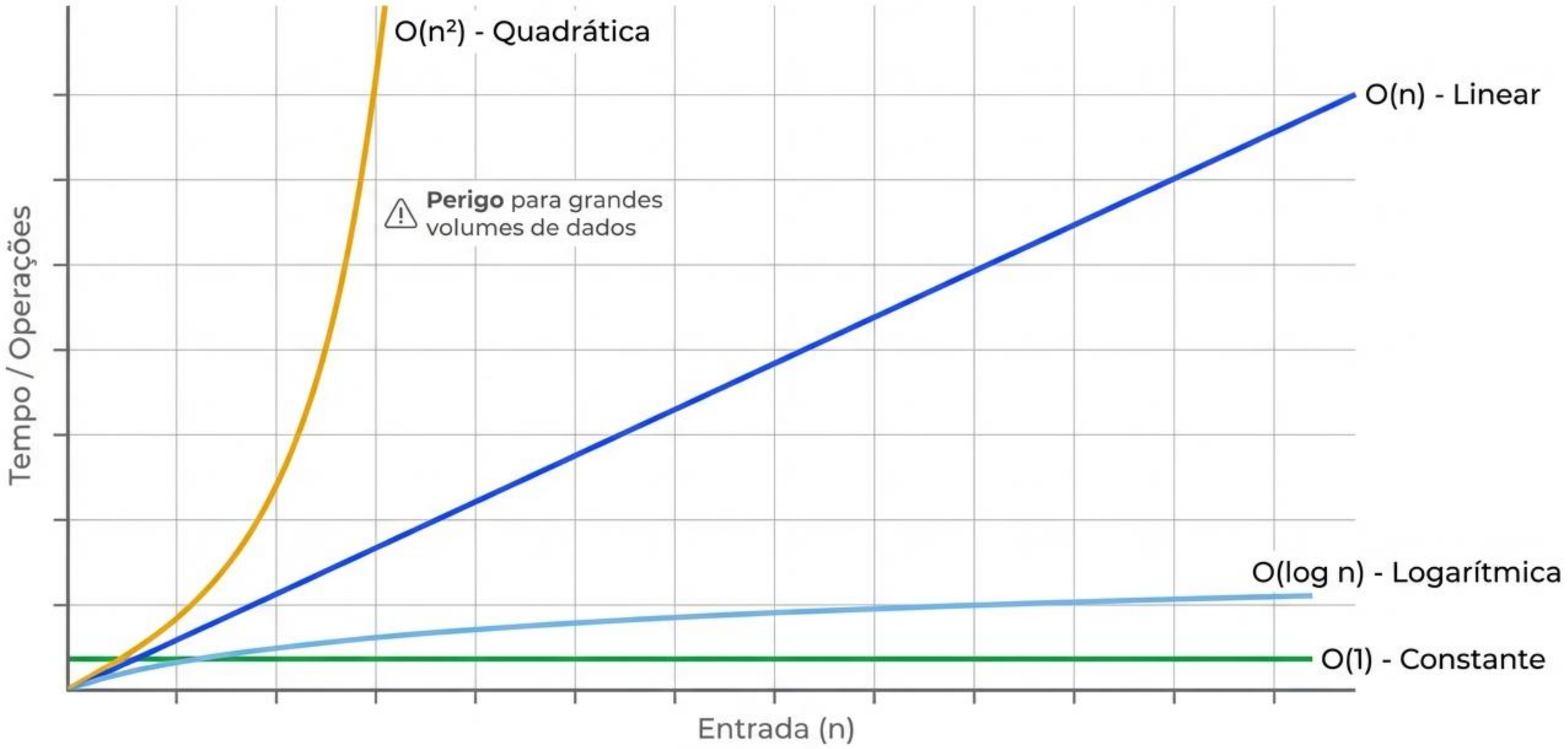
Impacto: Dobrar n quadruplica o tempo no cenário quadrático.

A Notação Big O (O)

- Definição: Limite Superior Assintótico (Upper Bound).
- Representa o Pior Caso (Worst Case Scenario).
- Garantia: O tempo de execução não excederá esta taxa.
- Referência: Padrão da indústria e literatura (Cormen).



Classes Fundamentais de Complexidade



Complexidade em Estruturas Python

Operação Python	Complexidade (Big O)
Acesso a índice lista[i] ou chave dict	O(1)
Busca simples "item in lista" ou loop for	O(n)
Ordenação eficiente (.sort() / Timsort)	O(n log n)
Ordenação ingênua ou loops aninhados	$O(n^2)$

O Veredito Final

Análise de Hardware, Limites e o Futuro do seu Código.

O Estouro da Pilha (RecursionError)

- Erro atingido em n elevado: `maximum recursion depth exceeded`.
- A memória RAM não é infinita; a Call Stack tem um teto.
- Trava de segurança padrão do Python (aprox. 1000 chamadas).

```
RecursionError: maximum recursion depth exceeded
```

Iterativo vs Recursivo: Os Dados

- **Tempo:** Iterativo escala melhor; Recursivo sofre overhead.
- **Espaço (Memória):** Iterativo O(1) vs Recursivo O(n).
- **O Pedágio:** Empacotar, empilhar e desempacotar contextos.

O Paradoxo da Recursão em Cauda

- Tail Recursion: Quando a recursão é a última operação da função.
- Algumas linguagens (C++, Scheme) otimizam isso para O(1) de memória.
- O Paradoxo Python: O interpretador padrão não otimiza Tail Recursion.

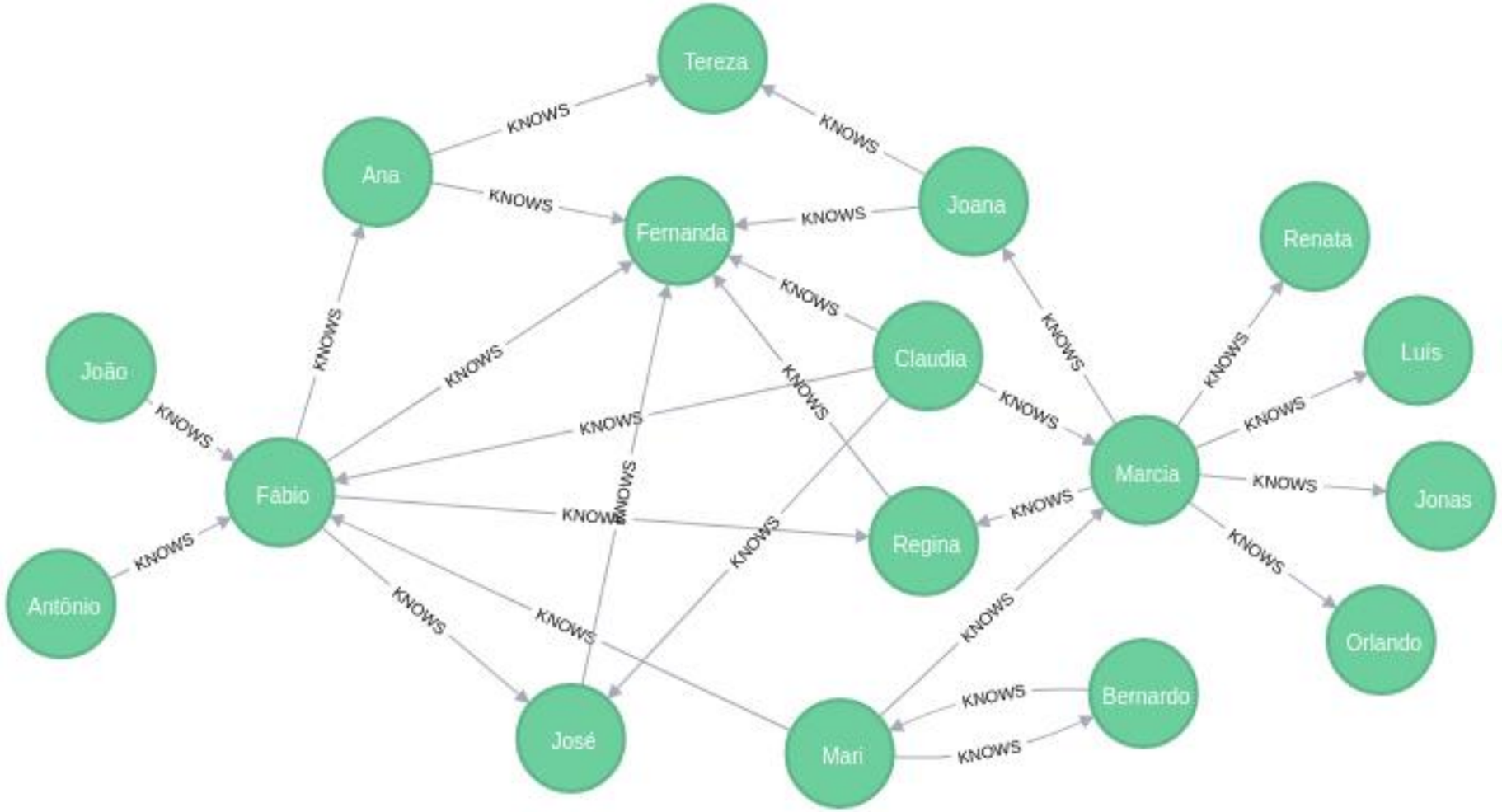
```
return tail_fat(n-1, n*acc)
```

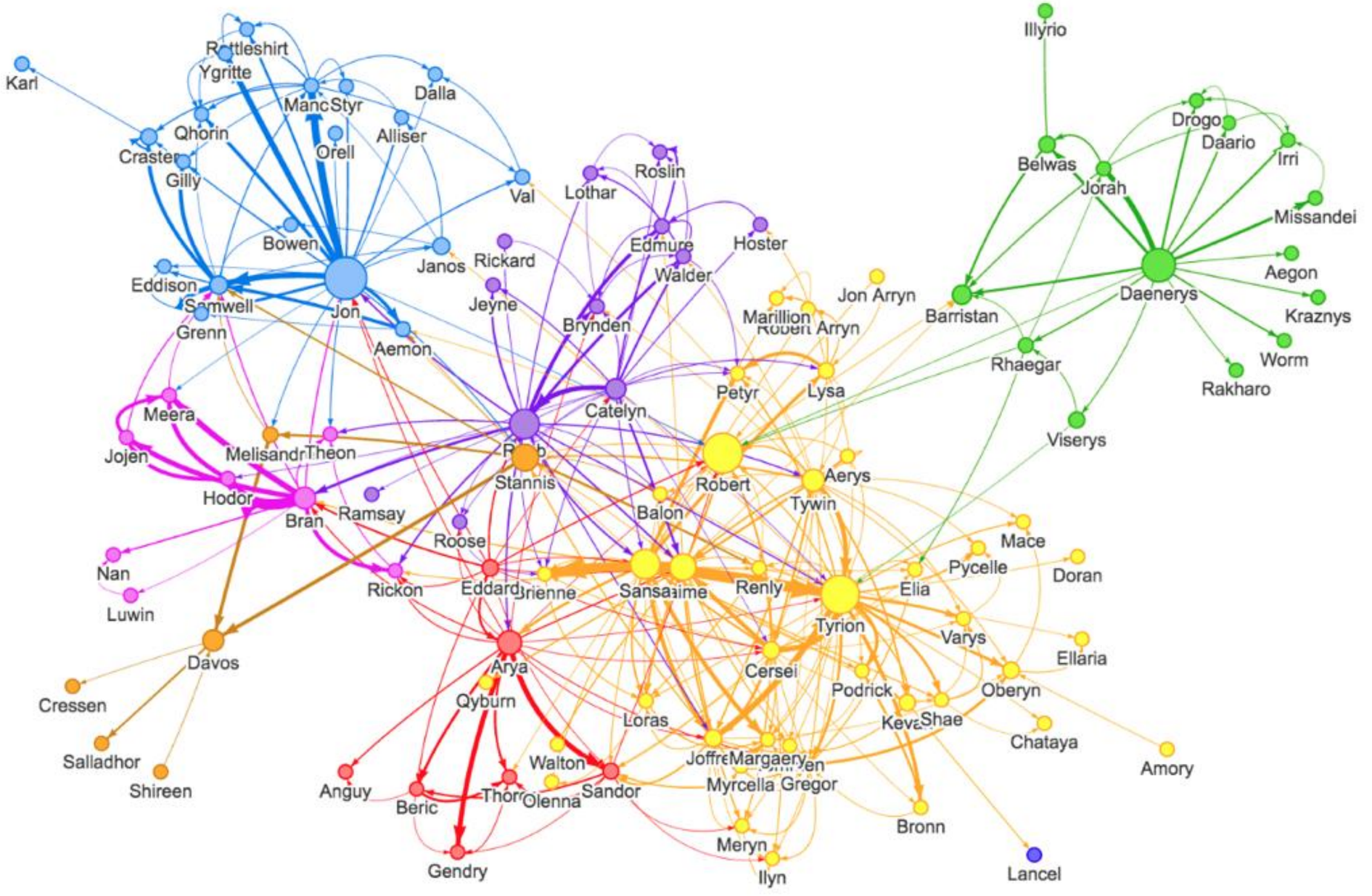
Check de Aprendizado

Se o iterativo é superior em tempo e memória no Python, por que nós ainda precisamos estudar a recursividade?

Escolhendo a Ferramenta Certa

- **Use Iterativo:** Performance crítica, estruturas lineares (Vetores, DataFrames).
- **Use Recursivo:** Elegância de código, estruturas ramificadas (Árvores, Grafos).
- **Estratégia:** Dividir e Conquistar (subproblemas).


Tereza
Ana
Joana
Fernanda
Renata
Claudia
Luís
João
Marcia
Jonas
Fábio
Regina
Antônio
Orlando
Bernardo
José
Mari
KNOWS


Illyrio
Karl
Rattleshirt
Ygritte
Dalla
MancStyr
Qhorin
Alliser
Orell
Craster
Gilly
Val
Roslin
Lothar
Drogo
Daario
Irri
Belwas
Jorah
Missandei
Bowen
Edmure
Hoster
Rickard
Janos
Walder
Aegon
Eddison
Samwell
Jon
Jeyne
Brynden
Jon Arryn
Marillion
Robert Arryn
Barristan
Daenerys
Kraznys
Grenn
Aemon
Rhaegar
Worm
Petyr
Lysa
Rakharo
Meera
Catelyn
Viserys
Jojen
Melisandr
Theon
Robb
Stannis
Robert
Aerys
Hodor
Tywin
Bran
Ramsay
Balon
Roose
Mace
Pycelle
Doran
Nan
Rickon
Eddard
Brienne
Sansa
Jaime
Renly
Elia
Luwin
Tyrion
Varys
Davos
Arya
Cersei
Ellaria
Podrick
Oberyn
Cressen
Qyburn
Loras
Shae
Chataya
Salladhor
Joffrey
Margaery
Walton
Amory
Shireen
Anguy
Beric
Thor
Olenna
Sandor
Myrcella
Gregor
Bronn
Meryn
Gendry
Ilyn
Lancel

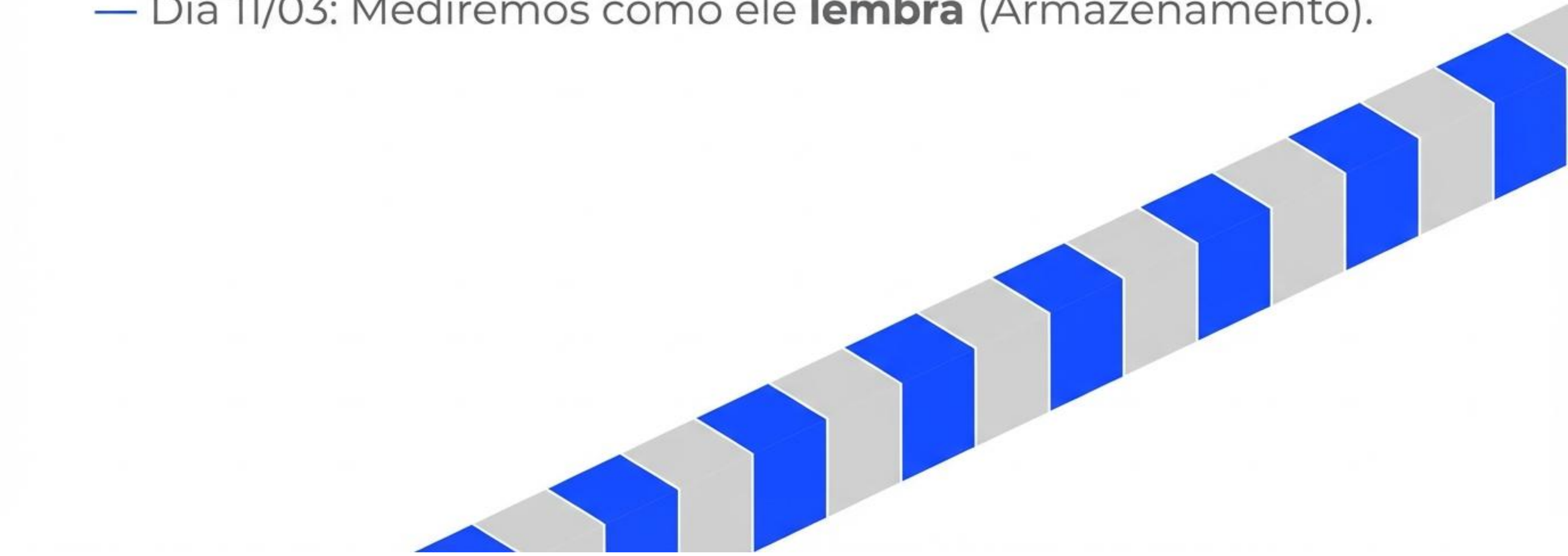
Garantindo os Pontos de Hoje

- Documente os tempos do seu hardware no notebook.
- Adicione uma docstring (PEP 8) justificando a vitória do iterativo.
- **Ação:** Faça o Push/Commit no seu repositório do GitHub.

Próxima Aula: Vetores e Matrizes

— Hoje: Medimos como o computador **pensa** (Processamento).

— Dia 11/03: Mediremos como ele **lembra** (Armazenamento).



Laboratório: Teoria vs. Prática

Objective: Medição empírica de tempo com biblioteca `time`.
Cenário: Implementar Fatorial Iterativo vs. Fatorial Recursivo.
Análise: Observar o overhead da recursão e o estouro de pilha (stack overflow).

Dúvidas?

CRIATIVIDADE
INOVAÇÃO
¡DEIA!
COMAMUNIDADE
CRIATIVIDADE
BRAINSTORM
INOVAÇÃO

ibmec

A direção do amanhã.

/ibmec